Calibrating Bug-Taxonomy Claims for Competitive-Programming Failures

Anonymous Author(s)
Submitted for review

Abstract*—Bug taxonomies are attractive for explaining failed generated code, but scalable studies usually label from weak evidence: problem statement, buggy code, and verdict. We ask which claims such labels can support. As a calibration case study, we build 14,182 sandbox-confirmed buggy C++ submissions for 107 post-cutoff CodeContests problems: human submissions, a cutoff-aligned proxy model, a descriptive modern model, and a public-example self-reflection loop. We reuse the 32-leaf taxonomy of Wei et al., collect a 375-bug doubly annotated gold set, validate an LLM judge, and audit a separate 120-item subset with first failing tests, actual/expected output, and reference solutions. The result is a claim ladder. Sandbox verdicts support label-free outcome claims. Validated family labels support coarse triage, but not small source-level differences: the judge reaches substantial agreement with the gold ($\kappa \approx 0.76$) while over-applying design by 6–9 points, and an arm-specific bias correction leaves focused source shifts small or order-unstable. Common-evidence labels do not establish root causes; even stronger-evidence labels remain a diagnostic construct rather than final proof. Stronger-evidence relabeling changes 55.0% of audited family labels, mixing evidence gain, taxonomy-boundary ambiguity, and harder-task instability. Applying this ladder, the only stable taxonomy signal in the case study is a broad math/design concentration; low-level syntax/compile failures remain a source-conditional separation and the cutoff proxy's 3.0% acceptance bounds generality. Label-free dynamics are clearer: public-example self-reflection repairs only 4.0% of proxy-model bugs, matching same-budget resampling and fixing no runtime or time-limit failures. We release the corpus, prompts, cached outputs, labels, and analysis scripts.

***Index Terms*—large language models, competitive programming, bug taxonomy, empirical study, LLM-as-a-judge, data contamination**

I. Introduction

Code-generating large language models (LLMs) have moved from toy benchmarks to competitive programming, a domain demanding precise algorithmic reasoning, careful edge-case handling, and strict conformance to input/output (I/O) formats [1], [2]. Most work measures *whether* models solve such problems, via pass@k on benchmarks like HumanEval, MBPP, and APPS [3]–[6]. Far less is known about *how* models fail, and even less about which failure claims are justified by the evidence available at scale.

The question matters because taxonomies are already used as measurement instruments. If a taxonomy identifies stable failure regions, it can guide review, testing, and education for AI-assisted development. If labels instead reflect missing tests, over-broad categories, or an automated judge's granularity, then a precise-looking table can mislead. Calibrating such claims is hard for three reasons. First, competitive-programming benchmarks suffer from *data contamination*: problems and editorials appear in pre-training data, so apparent failures may reflect memorization rather than reasoning [2], [7]. Second, a useful calibration needs the *same* problems and a common vocabulary across human and generated failures, while still avoiding a false behavioral equivalence between curated human submissions and fixed-budget model samples. Third, labeling tens of thousands of buggy programs into a fine-grained taxonomy is beyond manual reach, pushing studies toward automated (LLM) labeling whose reliability is itself unestablished.

We address all three. We adopt the recent bug taxonomy of Wei et al. [2]—six general-error and six algorithm-specific families, 32 leaves (Table I)—and bound contamination risk by using the test split of CodeContests [1]: 107 Codeforces problems first published after the September-2021 cutoff of `gpt-3.5-turbo` [8]. We use four sources as a stress test for the instrument: human submissions; a cutoff-aligned proxy model; a modern model whose training window covers the problems (a descriptive contrast, not a cutoff-controlled source); and a self-reflection loop on the cutoff proxy. Every submission is judged in a hardened sandbox; two independent annotators label a stratified 375-bug gold set; and an LLM judge labels the full corpus, which we validate against the human gold and the human inter-annotator ceiling.

We answer four research questions, ordered from measurement calibration to the case-study claims it permits:

- **RQ1:** Which taxonomy claims are reliable under common evidence, automated labeling, and stronger failure evidence?
- **RQ2:** Which calibrated taxonomy signals remain in the case study?
- **RQ3:** How do capability, contamination risk, and problem difficulty shape label-free dynamics?
- **RQ4:** What does public-example self-reflection repair, and which failures persist?

The main finding is a calibrated claim ladder. (1) A validated LLM judge is adequate for coarse family triage, but not for fine source separation: it reaches substantial agreement with the gold while inflating design labels by 6–9 points. (2) Common-evidence labels are diagnostic proxies, not root causes: first-failing-test/reference evidence changes 55% of audited family labels and still requires adjudication. (3) Fine source-family shifts are therefore exploratory screens; the defensible taxonomy signal in this case study is only coarse concentration in mathematics/design, plus a separate syntax/compile separation in model samples. (4) Label-free dynamics are stronger: the modern contaminated reference is more capable but difficulty-sensitive, and public-example feedback repairs only 4% of proxy-model bugs, matching same-budget resampling with no runtime or time-limit fixes.

Contributions: **(i)** a released case-study artifact for failed-submission measurement (14,182 bugs, 107 identical problems, 375 doubly-annotated gold items); **(ii)** a transferable claim-calibration checklist—human gold, power analysis, problem-level bootstrap, per-family judge error, arm-specific bias correction, and a stronger-evidence audit—that states what each evidence level can and cannot support; **(iii)** empirical evidence that common-evidence taxonomy labels are often not stable under stronger evidence; and **(iv)** label-free and taxonomy-aware self-reflection analyses. All artifacts are released.

II. Background and Related Work

A. LLMs for code and competitive programming

Transformer language models [9], [10] pre-trained on source code now span open and proprietary families [3], [8], [11]–[16], and a broad literature surveys their software-engineering uses [17]. Competitive programming is repeatedly singled out as the hardest target because it couples non-trivial algorithmic reasoning with strict resource and I/O-format constraints [1], [2]: AlphaCode [1] reached competition-level generation only through massive sampling and filtering. Most evaluation reports *whether* a model succeeds—pass@k on HumanEval, MBPP, APPS, and successors [3]–[5], [18], or repository-level tasks like SWE-bench [19], drawing on code corpora and contest archives [20]–[22]—and rigorous re-testing shows such pass rates are often optimistic [6]. We instead study what can be learned about *how* failures occur, and what evidence is needed before taxonomy labels become defensible measurements rather than diagnostic guesses.

B. Data contamination

Because contest problems and editorial solutions are public, they routinely enter pre-training corpora and inflate benchmark scores; a growing line of work measures and mitigates this [23]–[26]. LiveCodeBench [7] adopts rolling, recent problems and reports strong models near zero on the hardest unseen tasks, and Wei et al. [2] run an explicit contamination audit. We take the same stance structurally: the CodeContests test split post-dates the `gpt-3.5-turbo` cutoff (our cutoff proxy), and we add a modern model as a deliberately contamination-uncontrolled contrast rather than a cutoff-controlled measurement.

C. Bug taxonomies and empirical bug studies

Defect classification has a long lineage—Orthogonal Defect Classification [27], empirical bug characterizations [28]–[30], and curated fault benchmarks [31], [32] used to study repair [33]—almost all on *human* code; security studies of AI code (e.g., Copilot [34]) examine vulnerabilities rather than a general bug taxonomy. Closest to us, Wei et al. [2] recently proposed the taxonomy we adopt and hand-labeled 75 incorrect programs from a single model with strong inter-annotator agreement ($\kappa = 0.86$), using *no* automated labeler and providing *no* human baseline. We reuse their taxonomy verbatim but differ in kind as much as scale: we label **14,182** bugs (a $\sim$190$\times$ larger corpus) from *four* sources—including *human* submissions to the *same* problems—under explicit cutoff alignment and contamination caveats, with an LLM judge whose reliability we validate against a 375-bug doubly-annotated human gold. The human baseline lets us calibrate source effects on identical problems, and the scale forces measurement questions—statistical power, within-problem clustering, and labeler granularity—that a 75-program, single-model, AI-only study never has to confront.

D. Automated repair and self-correction

Automated program repair has matured from search- and learning-based methods [32], [33] surveyed broadly [35], [36] to LLM-based repair [37]. For models repairing their *own* output, Self-Refine [38] critiques and revises with self-feedback, self-debugging [39] and Reflexion [40] add execution or verbal feedback, and Olausson et al. [41] question whether self-repair is a "silver bullet." We instantiate a reflection loop with the public example-test results a contestant actually sees, and ask whether repair changes the *type* of remaining bugs rather than only the pass rate.

E. LLM-as-a-judge and agreement methodology

Using an LLM to score or label outputs is now widespread [42], [43], but it carries known biases [44] and its reliability for fine-grained, domain-specific taxonomies is rarely audited. We treat the judge as a measurement instrument: we size the human gold by Cochran's proportion formula [45], [46], report chance-corrected agreement and its interpretation [47]–[51], and follow empirical-software-engineering guidance [52], [53].

F. The taxonomy we adopt

We do not propose a taxonomy; we reuse Wei et al.'s [2] verbatim (Table I) so our labels are comparable to theirs. It has two top levels. *General Errors* (GE) describe *how* a program is wrong regardless of the algorithm: design/wrong-algorithm (GE1), boundary and off-by-one (GE2), condition and operator mistakes (GE3), data-type issues such as integer overflow (GE4), syntax/compile errors (GE5), and input/output formatting (GE6). *Algorithm-specific Errors* (AE) describe *which* algorithmic technique was misapplied: mathematics (AE1), greedy (AE2), graph (AE3), recursion/divide-and-conquer (AE4), dynamic programming (AE5), and search (AE6). Each of the 12 families is split into leaves—e.g. GE1 separates *Incorrect Algorithm*, *Misunderstanding Requirements*, and *Inefficient design*—for 32 leaves total. Labeling is multi-label, since one program can exhibit several independent errors. The GE/AE split is the axis our analysis returns to: GE5/GE4 are coding errors a tool can catch, whereas GE1 and the AE families are reasoning errors that require understanding the problem.

TABLE I
THE TAXONOMY OF WEI ET AL. [2]: 6 GENERAL-ERROR (GE) AND 6 ALGORITHM-SPECIFIC (AE) FAMILIES, 32 LEAVES.

Code	Family (#leaves)
GE1	Design: wrong algorithm, misread requirement (4)
GE2	Boundary: edge cases, off-by-one/indexing (2)
GE3	Condition: predicates, operator/precedence (2)
GE4	Data type: overflow, implicit conversion (2)
GE5	Syntax: compile errors, language misuse (2)
GE6	Input/Output: parsing, output formatting (2)
AE1–AE6	Math, greedy, graph, recursion/D&C, DP, search (18)

III. STUDY DESIGN

Figure 1 summarizes the pipeline: build a cutoff-filtered benchmark; collect human and AI buggy submissions; confirm verdicts in a sandbox; label a human gold sample and scale with an audited LLM judge; calibrate which claims the resulting labels can support.

A. Benchmark, sandbox, and submissions

We use the CodeContests [1] *test* split: Codeforces problems first published after 2021-09-21, conservatively after `gpt-3.5-turbo`'s September-2021 cutoff [8]. We treat this cutoff as necessary but not sufficient—the exact training window of the `-0125` snapshot is not publicly documented per-release—so we add a behavioral sanity check: the cutoff proxy's acceptance is low and *flat* across difficulty ($\leq 8\%$; Sec. IV-C), which is consistent with solving from the statement rather than recalling seen tasks. We also run a lightweight code-similarity audit against the stored human accepted solutions for each problem: none of the proxy model's 64 accepted outputs is whitespace-normalized identical to a stored human accepted solution, and the maximum token 5-gram Jaccard similarity is 0.47 (median 0.20). This does not prove absence of training exposure, but it rules out verbatim or near-verbatim replay of the accepted solutions available in our corpus. Throughout, we therefore call this arm the *cutoff proxy*, not a certified contamination-free model. Every cutoff-aligned taxonomy claim rests on this proxy; we make none on the modern model. We keep a problem only if it is judgeable (≥ 1 correct human solution confirmed AC and $\geq$10 incorrect ones confirmed non-AC), leaving **107 problems** (ratings 800–3500). The set is deliberately hard-skewed—25 problems rated $\leq$1199, 12 at 1200–1599, 13 at 1600–1999, 22 at 2000–2399, and 35 rated 2400+—so RQ3's difficulty analysis spans the full competitive range rather than only easy tasks. Each C++ submission is compiled with GNU `g++` (`gnu++17`) and run under an OS sandbox (no network, bounded CPU/memory/stack/file size, process-group timeouts) on public and private tests; output comparison is whitespace-tokenized, case-insensitive, 10^{-4} float tolerance. Each run yields one of five verdicts (Table II); when a submission fails differently on different tests we report the most severe under the precedence CE > RE > TLE > WA > AC (a program that does not compile cannot be timed, one that crashes cannot be judged for output, and so on), cached by `(problem, source, tests)`. A bug is any submission whose verdict is not AC.

TABLE II
SANDBOX VERDICTS. A *bug* IS ANY NON-AC SUBMISSION; THE VERDICT IS THE MOST SEVERE OUTCOME OVER ALL TESTS.

Verdict	Meaning
AC	Accepted: correct output within limits on every test
WA	Wrong Answer: output mismatches the expected output
TLE	Time-Limit Exceeded: exceeds the problem's CPU budget
RE	Runtime Error: crash, non-zero exit, or sandbox fault
CE	Compile Error: source fails to build with `g++`

TABLE III
BUG CORPUS OVER 107 PROBLEMS. THE SUCCESS METRIC IS NOT COMPARABLE ACROSS HUMAN AND AI: HUMAN IS A CURATED CORRECT:INCORRECT RATIO; AI IS PER-SAMPLE PASS RATE.

Source	Confirmed bugs	Success metric
Human	8,648	53.4%
`gpt-3.5` proxy	2,076	3.0%
`gpt-5.4-nano` modern	1,465	31.5%
`gpt-3.5` reflect	1,993	—
Total	14,182	

Human solutions come labeled in CodeContests; we re-judge all and keep only label-consistent ones, yielding **8,648 human bugs** (and 9,925 confirmed-correct). **AI** solutions are generated zero-shot from a PaR-style prompt [2] (task framing plus the verbatim statement, which carries the I/O format and examples) at temperature 1.0 with a *fixed* budget of 20 samples per problem (no early stop): `gpt-3.5-turbo-0125` (proxy) gives 64 AC and **2,076 bugs**; `gpt-5.4-nano` (modern) gives 675 AC and **1,465 bugs**. We stress that `gpt-5.4-nano` is *not* a cutoff-controlled source: its training window covers these problems, so it cannot isolate reasoning from memorization. It is included for a different purpose—to stress-test labeler and sample effects on a present-day arm—and `gpt-3.5-turbo-0125` alone carries the cutoff-aligned claims. Every cutoff-aligned taxonomy result in this paper rests on this proxy model; the modern model is descriptive context, read always against that caveat. **Self-reflection** applies up to three Self-Refine rounds [38] per proxy-model bug, continuing the same conversation with the public example-test results, stopping at AC; **1,993** bugs survive. The total is **14,182 sandbox-confirmed bugs** (Table III).

a) Estimand and non-goals: The human and AI corpora are not interchangeable behavioral samples. Human submissions are curated CodeContests artifacts with separate correct and incorrect pools; AI submissions are fixed-budget samples from a prompt. We therefore do not use the human correct:incorrect ratio and AI per-sample pass rate to compare programmer ability. The taxonomy estimand is narrower and conditional: *among sandbox-confirmed non-AC C++ submissions on the same problems*, how are bug mechanisms distributed? Acceptance rates are used only for

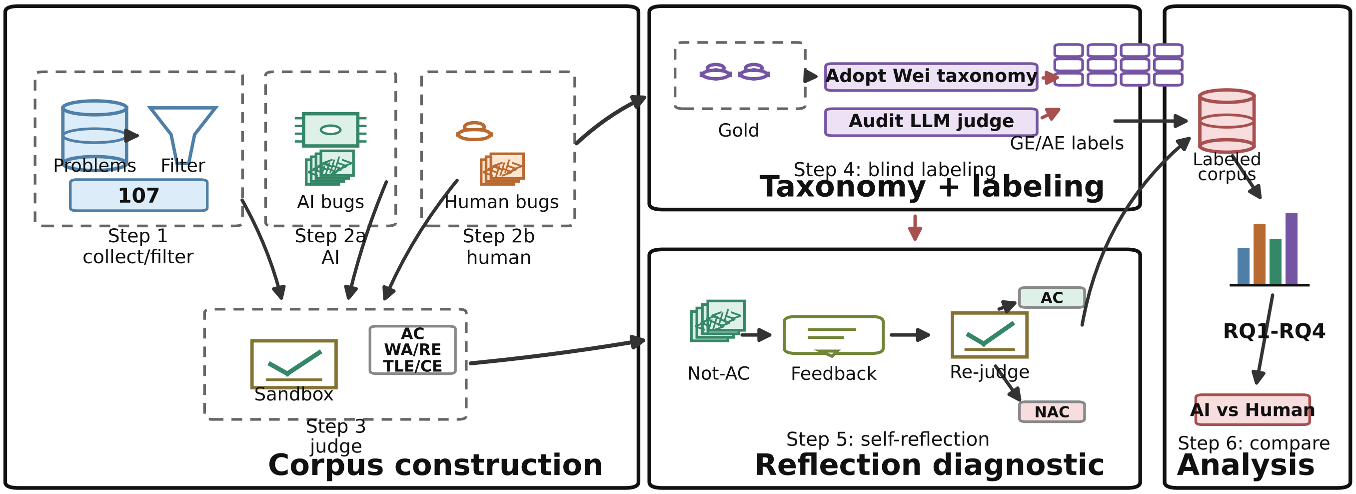

Fig. 1. Overview of the study pipeline: CodeContests problems are filtered into a post-cutoff C++ benchmark; human, zero-shot AI, and self-reflection submissions are judged by the same sandbox; taxonomy labels come from a fixed Wei et al. taxonomy through a blind human gold set and an audited LLM judge; claim strength is calibrated against stronger-evidence relabeling.

within-source trends (e.g., by difficulty) and for the proxy-vs-modern contamination/capability contrast. This estimand does not claim that a curated human incorrect submission and a zero-shot LLM sample arise from the same behavioral process.

B. Prompts and protocols

We summarize the three prompt-driven protocols; full prompts are released. *Generation* follows the PaR template [2]: a system message frames the task ("write a correct, efficient C++ program; return only a code block"), and the user message is the verbatim problem statement, which on Codeforces already contains the I/O specification and worked examples; no unit tests or scaffolding are added, so the model must reason from the statement alone. The fixed-budget choice—exactly 20 samples per problem, no early stop—equalizes effort so that a model's bug yield reflects its true failure rate rather than a collection cap; the proxy model yields ≈19 bugs per problem, the modern model ≈14. *Self-reflection* continues the same conversation: we append the model's own buggy program as the assistant turn, then a user turn reporting its results on the public example tests (the input, the expected output, and the program's actual output for each), and ask for a corrected program, for up to three rounds, stopping at the first AC. The model thus sees exactly the feedback a contestant reads from the sample tests, and nothing more. *Judging* gives `gpt-5.4` the problem statement, the buggy code, and the verdict, and asks for the applicable taxonomy leaves plus a one-line rationale in strict JSON, sampled three times with majority aggregation. All model calls use the OpenAI Chat Completions API through the model strings named above; generation/reflection use temperature 1.0, the judge uses temperature 0.4, and no `top_p`, frequency, or presence penalties are set. Responses are cached by model, temperature, prompt, and nonce; because proprietary model strings are not immutable snapshots, the released raw generations, judge outputs, and cache keys define the audit record. The API calls that produced the released generation/reflection corpus were completed on 2026-06-29, and the judge labels on 2026-06-30. The cache stores raw response text rather than provider response identifiers, so exact re-generation is not promised; reproducibility means re-running the analysis from the released outputs. The judge is provenance-blind and receives neither the failing hidden test nor a reference solution, matching the human annotators' information exactly so that disagreement reflects judgment, not evidence. This is a deliberate choice for an apples-to-apples agreement study, but we acknowledge its cost: inferring *which* algorithmic family is at fault for a wrong answer from code alone is genuinely hard, and that irreducible diagnostic noise is itself a plausible contributor to the fine-grained instability we document in RQ1. We therefore keep the common-evidence protocol for the full corpus, and add a separate first-failing-test audit to measure how much stronger evidence changes labels.

C. Labeling and analysis

Labeling is multi-label into the 32 leaves [2]. We size the gold by Cochran's formula with finite-population correction [45], [46]: 95% confidence, ±5% margin on 14,182 items gives $n \approx 375$. We allocate it arm-balanced (125 each: human, proxy, modern), stratify by verdict so rare verdicts appear, and cap four items per problem. Annotators used the fixed taxonomy with the same decision rules in both gold and audit packs: prefer the most specific leaf that explains the demonstrated failure, multi-label only for independent causes, and do not label downstream symptoms. Primary-family tables use the first leaf after this rule. **Two annotators** labeled all 375 items provenance-blind (problem statement and buggy code only); a third adjudicated; we report their inter-annotator agreement. The **LLM judge** (`gpt-5.4`) labels all 14,182 bugs from the same evidence (statement, code, verdict) with self-consistency over three samples. For verdict-based questions (acceptance, difficulty, repair) we use the exact full corpus; for taxonomy-distribution questions we use the human gold and validate the judge against it (RQ1). We report a descriptive χ^2/Cramér's V on family *occurrences*: each multi-label bug contributes one count to every family present. Nine families appear at least once in the zero-shot corpus (GE1–GE6, AE1, AE2, AE5), giving $\mathrm{df} = 8$ for pairwise source comparisons; AE3/AE4/AE6 are absent and the GE3 singleton is kept in the test but omitted from the compact table. We do not use this anti-conservative test for inference. Instead, we use a problem-level bootstrap because bugs are multi-label and clustered within problems. For each replicate we sample 107 problem IDs with replacement; within each sampled problem and source, we compute the share of that source's non-AC bugs whose labels contain each family, skipping problem-arm cells with no bugs; then we average the model–human difference over sampled problems. We report percentile 95% intervals from 2,000 replicates. This estimates an equal-problem conditional bug-profile difference, rather than letting high-volume problems dominate the estimate.

Three design choices in this protocol are worth making explicit, because they determine what the calibration can support. First, the gold is *arm-balanced* rather than proportional: drawing 125 bugs from each source—not 375 in proportion to the 8,648/2,076/1,465 corpus split—trades population-level precision for equal power to characterize each source's profile. Second, *verdict stratification with rare-verdict floors* guarantees that compile, runtime, and time-limit failures appear in the gold despite being a few percent of the corpus, so the rare-but-diagnostic families (syntax GE5, and the structural failures central to RQ4) are estimable at all. Third, labeling is *provenance-blind by construction*: the annotator and judge packages contain the statement, the code, and the sandbox verdict, with the author identity stripped and the within-problem order shuffled, so neither a human nor the judge can infer "this looks like AI code" and label to a stereotype—the protocol measures the bug, not a guess about its origin. Statistically, we treat the judge as an instrument whose error we bound against the human ceiling (RQ1) rather than as ground truth, and we report effect sizes alongside p-values throughout because, at n=125 per arm and with within-problem clustering, significance and magnitude carry different and complementary information.

Finally, after the main analysis, two annotators independently re-labeled a stratified 120-item wrong-answer audit set (40 per zero-shot source) with stronger evidence: the same statement and code plus the first failing test's input, expected/actual output, and one accepted reference implementation. If a program failed a public sample first, that public sample is the first failing test; otherwise the evidence is private-test based (95 public, 25 private overall: 18/40 public for human, 40/40 for proxy, 37/40 for modern). The n=120 audit estimates the overall family-change rate with about $\pm$9 percentage points of binomial precision; it is not designed for per-family prevalence. The annotators adjudicated all disagreements, with a third tie-breaker for three items. This audit does not relabel the full corpus; it estimates how far common-evidence diagnostic labels move when the same taxonomy is applied with stronger evidence.

TABLE IV
JUDGE MEASUREMENT-ERROR SENSITIVITY. P/R USE THE 315 GOLD ITEMS WITH ANNOTATOR-AGREED PRIMARY FAMILIES. CORRECTED DELTAS SUBTRACT EACH ARM'S JUDGE-MINUS-HUMAN BIAS MEASURED ON THE 375-ITEM GOLD.

Family	P/R	Proxy–H	Modern–H
GE1 Design	74.5/96.3	-6.7→-5.1	+0.9→-1.9
AE1 Math	95.7/80.0	+0.5→+0.9	-2.0→+6.8
GE2 Boundary	76.9/95.2	+1.7→+1.7	-1.9→-4.3
GE6 I/O	100.0/66.7	-1.2→-2.8	-1.2→-2.0
AE1+GE1	–	-6.2→-4.2	-1.1→+4.9

TABLE V
LABELING AGREEMENT ON THE 375-BUG COMMON-EVIDENCE GOLD (COHEN'S κ, MICRO-F_1).

Pair	κ (leaf)	κ (family)	micro-F_1
human1 vs. human2 (ceiling)	0.80	0.83	0.80
judge vs. human1	0.75	0.77	0.76
judge vs. human2	0.77	0.81	0.77

IV. RESULTS

A. RQ1: Which taxonomy claims are reliable?

The first result is about the instrument, not the model. The two human annotators on the 375-bug common-evidence gold agree strongly—Cohen's $\kappa = 0.80$ at leaf level and 0.83 at family level—matching the high agreement reported for this taxonomy by Wei et al. [2]. The `gpt-5.4` judge reaches substantial agreement with both annotators (family $\kappa = 0.77/0.81$, micro-$F_1 \approx 0.77$; Table V), so it is usable for coarse full-corpus screening. It is not neutral enough for fine distributional claims. On the same 375 gold items the judge assigns the broad design family to 26/34/41% of human/proxy/modern bugs, while the human annotators assign 20/28/32%: a 6–9 point inflation that is arm-dependent and largest on the modern arm, which shares the judge's model family.

Table IV makes this error structure explicit rather than relying on aggregate κ: GE1 has high recall but lower precision, AE1 has the opposite shape, and additive arm-specific bias correction leaves the focused family shifts small or direction-unstable. The combined AE1+GE1 region remains large under correction (human 68.8%, proxy 64.6%, modern 73.7%), supporting coarse concentration but not an ordered source comparison.

TABLE VI

FIRST-FAILING-TEST/REFERENCE-SOLUTION RELABELING AUDIT ON 120 ZERO-SHOT WA SUBMISSIONS (40 PER SOURCE). THE TOP BLOCK IS INTER-ANNOTATOR AGREEMENT WITHIN THE STRONGER-EVIDENCE TASK; "CHANGED" ROWS COMPARE THE ADJUDICATED STRONGER-EVIDENCE LABEL WITH THE ORIGINAL COMMON-EVIDENCE LABEL.

Metric	Count	Value
Leaf exact agreement	84/120	70.0%
Family exact agreement	85/120	70.8%
Primary-family κ	–	0.69
Binary family κ	–	0.71
Adjudicated disagreements	36/120	30.0%
Leaf changed	70/120	58.3%
Family changed	66/120	55.0%
Math/design before	44/120	36.7%
Math/design after	58/120	48.3%
Into math/design	25/120	20.8%
Out of math/design	11/120	9.2%

TABLE VII

PRIMARY-FAMILY TRANSITIONS IN THE 120-ITEM STRONGER-EVIDENCE AUDIT. ROWS SHOW ORIGINAL COMMON-EVIDENCE FAMILY; DESTINATIONS LIST THE LARGEST STRONGER-EVIDENCE FAMILIES AFTER ADJUDICATION. SINGLETON ORIGINAL GE3 IS OMITTED.

Orig.	n	Same	Main destinations
GE1	21	11	AE1 6; AE6/GE3/AE5/GE2 1 each
GE2	20	2	GE1 8; GE3 5; AE1 3
GE4	10	1	AE1 5; GE2 2
GE6	8	2	GE1 4; AE1/GE2 1 each
AE1	22	15	AE5 4; GE2 2
AE2	25	19	GE3 3; GE1 2
AE5	13	7	GE1 3; GE2 2

TABLE VIII

CLAIM-CALIBRATION CHECKLIST USED BEFORE INTERPRETING RQ2–RQ4. THE STEPS ARE TRANSFERABLE; THRESHOLDS ARE EVIDENCE-LEVEL GATES FOR THIS STUDY, NOT UNIVERSAL SIGNIFICANCE CUTOFFS.

Claim	Rule	Outcome
Verdicts/repair	Deterministic	Direct claims
Coarse families	Labeler + audit	Bounded AE1/GE1
Fine shifts	Bootstrap + bias	Screen only
Root causes	Paired evidence	Not inferred

The stronger test is evidence stability. In the 120-item audit, annotators relabeled wrong-answer submissions with the first failing test, actual/expected output, and an accepted reference implementation. They again reached substantial agreement (primary-family $\kappa = 0.69$; binary multi-label family $\kappa = 0.71$), then adjudicated all 36 disagreements. Against the original common-evidence labels, the adjudicated stronger-evidence labels changed 70/120 leaves (58.3%) and 66/120 families (55.0%; Table VI). The rate is not confined to one source: family labels changed for 65.0% of human submissions and 50.0% of both AI arms.

Table VIII is the construct boundary for the rest of the paper. A full-corpus taxonomy difference is not promoted beyond a screen unless it survives problem-level clustering and the arm-specific bias correction in Table IV; root-cause claims are never made from common evidence alone. Common-evidence labels can say that a failure looks mathematical, design-like, or syntactic under the evidence a scalable study can usually afford. They cannot, by themselves, establish the root-cause mechanism of a wrong answer. The 55.0% should not be read as a pure evidence effect: the stronger-evidence task is itself harder, with lower annotator agreement than the common-evidence gold (family $\kappa = 0.69$ vs. 0.83), so the number combines evidence gain and root-cause-labeling instability. Nor do we claim the stronger-evidence label is uniquely true: one failing test and one reference solution can still leave multiple plausible fixes. The audit's role is to stress-test label stability under better evidence, not to certify final root causes. The transition summary (Table VII) shows where this instability concentrates: boundary, data-type, and I/O diagnoses are often reinterpreted as design, condition, or mathematics, while AE1 and AE2 are more stable. The audit also checks whether our coarsest claim survives stronger evidence. Its sample was deliberately stratified rather than population-representative, so it cannot estimate the full-corpus prevalence of math/design bugs; it only checks whether stronger evidence obviously erodes that bucket. Membership in {mathematics, design} is stable for 84/120 items (70.0%), and the crossings are net inward: 25 items move into the bucket and 11 move out. Family-set changes are not a single-arm artifact (65.0% human, 50.0% proxy, 50.0% modern) or a hard-problem artifact (65.2/50.7/57.7% for easy/medium/hard), but they are more common when the first failing evidence is private rather than public (72.0% vs. 50.5%). We therefore treat fine shifts as exploratory screens and reserve strong claims for bounded coarse patterns and label-free verdict dynamics.

B. RQ2: Which calibrated taxonomy signals remain?

RQ2 applies the calibration in Table VIII: we treat family distributions as screens unless a signal is larger than known labeler bias and survives the stronger-evidence audit. On the human gold (Table XI) all three sources are dominated by the same two families—mathematics (AE1, ≈33%) and design (GE1, 20–32%)—and their high-level general-vs-algorithmic split is similar without being diagnostic. A *weak* directional tendency is visible: design (GE1) runs 20% (human) / 28% (proxy) / 32% (modern), while implementation boundaries (GE2: 11/7/5%) and I/O formatting (GE6: 6% vs. <1%) run the other way. But the tendency is, on the gold, *underpowered*. At 125 bugs per arm the three-way family χ^2 can detect only $V \gtrsim 0.26$ at 80% power (a minimum-detectable-effect we compute by inverting the noncentral χ^2), and the observed $V \approx 0.23$ ($p \approx 0.07$–0.10 per annotator) sits just below that floor—so the gold can neither confirm the tendency nor rule it out, and we do not read its non-significance as evidence of no effect.

a) Full-corpus shifts are small, not self-interpreting: The full corpus helps with precision but not automatically with validity. A descriptive χ^2 on the source×family occurrence table over the nine observed families (Sec. III; df = 8) gives high significance at a negligible effect size—Cramér's $V = 0.11$

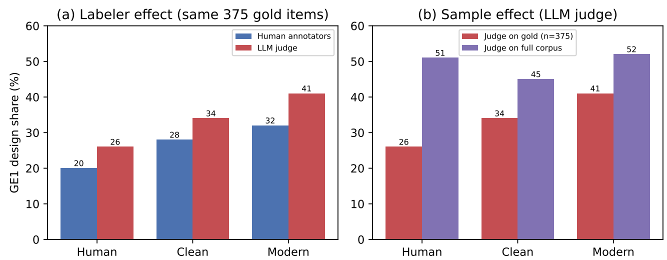

Fig. 2. Deconfounding the apparent design (GE1) reversal. **(a)** Holding the *sample* fixed (the same 375 gold items), the LLM judge inflates the design family (∼+6–9 points) but *preserves* the human<proxy<modern ordering—its granularity bias is a level shift, not a reversal. **(b)** Holding the *labeler* fixed (judge), moving from the stratified gold to the full corpus changes the ordering (proxy drops to last): the "reversal" is a sample effect. Neither route yields a robustly ordered difference.

(human–proxy, $p = 3 \times 10^{-25}$), 0.08 (human–modern)—and the problem-level bootstrap does detect several small shifts (Table IX). The proxy model has fewer design labels than humans (-5.6 points) and more mathematics/syntax labels (+4.0/+2.6); the modern model shows smaller shifts, with syntax and mathematics up by about 2–3 points and boundary/I/O down by about 1–2 points. These are not large taxonomy-defining separations: all major shifts are within the 6–9 point design inflation that RQ1 finds in the judge itself, and most are much smaller. We therefore use the judge-labeled full corpus as a *screen* for fine shifts, not as a causal claim that curated human submissions and fixed-budget model samples have different diagnostic distributions. The bias-corrected check in Table IV preserves only the bounded result: AE1+GE1 remains the dominant region for every arm, but source order is not stable. The low-level story is more qualified: data-type and I/O labels are rare everywhere, while syntax/compile failures remain a source-conditional separation.

b) The apparent reversal is a sample effect, not a labeler effect: A natural worry is that the comparison reverses under the judge: design runs 20/28/32 (human/proxy/modern) under the human annotators but 51/45/52 under the judge on the full corpus, the proxy model dropping to last. Holding the *sample* fixed dispels this (Fig. 2). On the same 375 gold items the judge gives 26/34/41—it *inflates* design by ∼6–9 points (a real granularity bias, quantified in RQ1) but *preserves* the human<proxy<modern ordering. The reversal appears only when the judge moves from the stratified gold to the natural corpus ($26/34/41 \rightarrow 51/45/52$), so it is driven by the *sample* (verdict-stratified, per-problem-capped gold vs. the full corpus), not the labeler. Neither sample nor labeler yields a stable ordered design story, which is why we treat the fine shifts as measurement-limited rather than explanatory.

The one structural regularity worth stating is what is *shared*: across both the gold and the full corpus, and across all three sources, the mass concentrates in the two "thinking" families (mathematics AE1, design GE1). This should not be mistaken for equality in low-level behavior. Syntax labels are higher for AI in the full corpus (GE5, +2.6/+2.9 points), and compile failures are more than twice as common for the models as for humans (Table XIII). Thus the careful statement is: low-level errors are minority modes for every source, but syntax/compile remains one of the clearest residual separations.

Nor does the coarse pattern eliminate the proxy-model floor confound. The cutoff proxy accepts only 3.0% of attempts, so many failures may be far from a near miss. We therefore report robustness checks as scope checks, not proof of equivalence. Table X shows that the AE1+GE1 concentration survives the WA-only slice and the stricter AC-problem slices, where a model has at least one accepted sample on the same problem. But the proxy AC-problem slice contains only 208 proxy bugs on easier problems, so it cannot rescue a fine source-comparison claim. It only supports the bounded statement that the defensible slices keep most mass in math/design rather than low-level coding families.

C. RQ3: Capability, contamination, and difficulty

Table III reports three success metrics, but they answer different questions: the human number is a curated correct:incorrect ratio, whereas the AI numbers are fixed-budget per-sample pass rates. We therefore do not use the table as a cross-source ability comparison. The defensible signal is the *within-source* trend with difficulty (Table XII, Fig. 3). The proxy model is at the floor everywhere ($\leq 8\%$). The modern model is much stronger but not simply solved by lookup: it falls from 55.0% on the easiest band to 12.3% on the hardest, with a non-monotone middle band (21.2% then 35.0%) that points to problem-set heterogeneity rather than a smooth ability curve. Its high acceptance on some hard problems is consistent with partial contamination, but the overall decline argues against pure memorization. The human rate is flat (42–59%), a survivorship artifact: harder problems attract stronger solvers.

The contrast between the two models is the point of the contamination design. The proxy model, whose cutoff predates every problem, never rises above 8% acceptance and is flat across difficulty—the behavior expected of a 2023-class model attempting unseen contest tasks from the statement alone. The modern model, whose training window covers the problems, scores far higher and declines overall, though not monotonically, with difficulty; this profile is hard to reconcile with pure lookup. Two readings are consistent with the data—real capability gains since the proxy model, and partial memorization of the easier, more-discussed problems—and we do not claim to separate them, only to show that an aggregate pass rate would conflate them whereas a difficulty-resolved one begins to pull them apart. The proxy model remains a cutoff-aligned proxy anchor for distributional claims; the modern model is the contrast that makes the anchor's value legible.

The verdict mix is exact and label-free (Table XIII). All sources are wrong-answer dominated (90–95%), but AI fails to *compile* more often than humans (proxy 6.1%, modern 5.1% vs. 2.6%) and the proxy model crashes more (RE 2.6%); the modern model has the fewest runtime errors (0.5%), consistent with cleaner low-level coding as capability grows.

TABLE IX

FULL JUDGE-LABELED CORPUS. PERCENT COLUMNS ARE CORPUS SHARES OF BUGS WITH A FAMILY PRESENT. Δ COLUMNS ARE EQUAL-PROBLEM MODEL–HUMAN DIFFERENCES WITH PROBLEM-BOOTSTRAP 95% CIS (2,000 REPLICATES). JUDGE-LABEL UNCERTAINTY IS HANDLED SEPARATELY BY THE BIAS SENSITIVITY IN TABLE IV; UNDER TABLE VIII, THESE ROWS REMAIN SCREENS.

Family	Human	Proxy	Modern	Δ Proxy–H	Δ Modern–H
GE1 Design	51.4	44.8	52.3	$-5.6[-7.9, -3.4]$	$-0.7[-3.9, +2.3]$
GE2 Boundary	8.2	9.9	6.3	$-0.6[-2.0, +1.0]$	$-2.3[-4.0, -0.4]$
GE4 Data type	1.0	0.1	0.1	$-0.7[-1.5, -0.1]$	$-0.7[-1.5, -0.1]$
GE5 Syntax	2.6	6.1	5.1	$+2.6[+0.8, +4.6]$	$+2.9[+0.7, +5.4]$
GE6 I/O	1.2	0.0	0.1	$-1.5[-2.0, -1.0]$	$-1.3[-1.9, -0.7]$
AE1 Mathematics	19.4	19.9	17.4	$+4.0[+2.3, +5.8]$	$+2.4[+0.9, +4.0]$
AE2 Greedy	7.7	9.2	10.1	$+0.4[-0.4, +1.3]$	$+0.8[-0.0, +1.5]$
AE5 DP	8.5	10.1	8.7	$+1.3[+0.4, +2.4]$	$-0.9[-2.9, +0.7]$
Cramér's V: H–C 0.11, H–M 0.08, C–M 0.09					

TABLE X

SCOPE CHECKS FOR THE COARSE AE1+GE1 CLAIM. VALUES ARE % OF BUGS IN MATHEMATICS OR DESIGN; PARENTHESIZED COUNTS ARE BUGS IN THE SLICE. AC-PROBLEM SLICES KEEP ONLY WA BUGS FROM PROBLEMS WHERE THAT MODEL HAS AT LEAST ONE ACCEPTED SAMPLE.

Slice	Human	Proxy	Modern
All non-AC	70.8 (8648)	64.6 (2076)	69.7 (1465)
WA only	73.8 (8181)	70.4 (1868)	73.6 (1370)
Proxy AC-problems	85.4 (874)	82.7 (208)	–
Modern AC-problems	75.6 (5083)	–	72.0 (779)

TABLE XI

BUG-FAMILY DISTRIBUTION ON THE HUMAN GOLD (% OF LABELS, MEAN OF TWO ANNOTATORS); BOLD MARKS THE LARGER HUMAN/AI SIDE. THIS IS A HUMAN-LABELED GOLD-SAMPLE TENDENCY, NOT A DEFINITIVE POPULATION ESTIMATE.

	Family	Human	Proxy	Modern
GE1	Design / wrong algorithm	20	28	**32**
GE2	Boundary / edge cases	**11**	7	5
GE5	Syntax	9	8	9
GE6	Input/Output	**6**	1	1
AE1	Mathematics	32	35	33
AE2	Greedy	8	**13**	9
AE5	Dynamic programming	8	6	8

TABLE XII

PER-SUBMISSION ACCEPTANCE (%) BY DIFFICULTY BAND.

Difficulty	Human	Proxy	Modern
$\leq$1199	59.3	7.6	55.0
1200–1599	49.5	0.8	43.8
1600–1999	42.9	0.4	21.2
2000–2399	55.5	3.6	35.0
2400+	53.3	1.0	12.3

TABLE XIII

VERDICT DISTRIBUTION PER SOURCE (% OF CONFIRMED BUGS, EXACT).

Source	WA	TLE	RE	CE
Human	94.6	0.9	1.8	2.6
Proxy	90.0	1.3	2.6	6.1
Modern	93.5	0.9	0.5	5.1

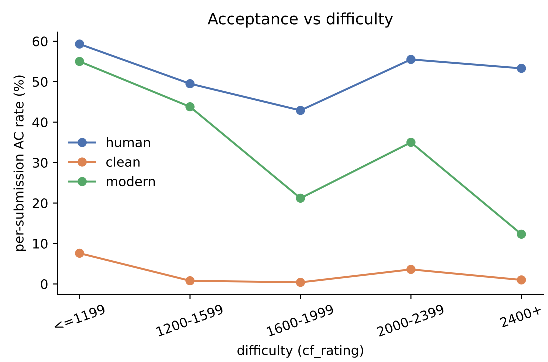

Fig. 3. Acceptance vs. difficulty: the proxy model is at the floor, the modern model declines overall but not monotonically, and humans are flat (survivorship).

That compile failures survive at all is itself notable: a 6% compile rate under a fixed prompt means the weak model periodically emits C++ that does not build, a failure mode essentially absent from curated human submissions and one that the self-reflection loop only partly clears (RQ4).

D. RQ4: What does self-reflection repair?

We study one concrete, common setup—the proxy model revising its own code with *public example-test* feedback for three rounds—and report what it can and cannot fix; the numbers below are specific to this model and this feedback signal, not a claim about self-reflection in general. Part of why the rate is so low is that the public examples were already in the original prompt (they ship with the Codeforces statement); reflection's *new* information is therefore only the model's own wrong output on inputs it had already seen, not fresh test data—so it can correct a mis-handled visible case but is blind to everything the hidden tests check. In that setting, repair reaches only **4.0%** (83/2,076) of the proxy model's bugs, and that number is not distinguishable from a trivial same-budget alternative. Using the existing 20-sample zero-shot budget as a leave-one-out pool, three fresh draws from the other 19 cached samples for each failed attempt would fix **82.6** bugs in expectation (**4.0%**; problem-bootstrap 95% CI 41.6–132.2); the reflection count lies inside that uncertainty range. Table XIV therefore changes the interpretation of RQ4: public-example reflection is not an efficient repair strategy

TABLE XIV
SELF-REFLECTION VS. SAME-BUDGET RESAMPLING FOR PROXY-MODEL BUGS. RESAMPLING ROWS ARE LEAVE-ONE-OUT EXPECTATIONS FROM EACH PROBLEM'S EXISTING 20 CACHED ZERO-SHOT SAMPLES, WITH PROBLEM-BOOTSTRAP 95% CIS; THEY ARE NOT NEW MODEL CALLS.

Strategy	Extra calls	Fixed	Rate
Self-reflection	3 repair	83	4.0%
Fresh sample	1	37.6 [17.5,61.9]	1.8%
Fresh samples	2	63.3 [29.6,101.5]	3.0%
Fresh samples	3	82.6 [41.6,132.2]	4.0%

over simple resampling; it is a diagnostic probe for which bugs survive weak feedback. A post-hoc edit audit of the 83 fixed trajectories agrees: 7 fixes are compile-error repairs, 16 leave the original program at least 95% character-identical, 47 occur on ≤900-rated problems, and three problems account for 46 fixes. Figure 4 tracks every bug's verdict across the three rounds: the accepted band grows $0 \to 53 \to 73 \to 83$, so most repairs land in the first round and the third adds only ten, while the wrong-answer mass (≈1,850) is essentially static. The rate is sharply stratified by verdict: compile errors reach 5.5% and wrong answers 4.1%, but **runtime (0/55) and time-limit (0/27) errors are never repaired**—runtime errors even tick up as the model perturbs code while chasing a wrong answer. Structural failures—a wrong-complexity algorithm, a crash on an unseen input—pass the small public examples unchanged, so reflection sees no signal. Because RQ1 validates automated labels only at family level, Fig. 5 uses the judge's primary family labels over all 2,076 proxy-model bugs. The result is still narrow: repairs concentrate in easy syntax/design cases, while boundary, mathematical, greedy, and DP failures mostly persist. The surviving diagnostic-label profile is therefore almost unchanged (general-error share 61%→61%): the AI's failure signature is *robust to self-correction*.

a) Repair power collapses with difficulty: Splitting the same trajectories by problem difficulty and original bug family (Fig. 5) shows the repair rate falling monotonically with rating: **7.1%** on easy problems (≤1599), **3.4%** on medium (1600–2399), and **1.4%** on hard (2400+). On hard problems the accepted band barely moves after the first round ($8 \to 10 \to 10$ of 693), whereas easy problems keep accruing fixes ($30 \to 43 \to 50$ of 700). Reflection thus helps exactly where the model was already close—easy problems with surface slips—and is nearly inert on the hard, design-bound problems where its bugs concentrate (RQ2, RQ3). Self-correction is not a substitute for choosing the right algorithm. Two secondary dynamics point the same way: compile errors—the one locally diagnosable failure—are cleared in every band (e.g. $54 \to 26$ on hard), but on hard problems they convert to *wrong answers* rather than acceptances; runtime errors slightly *increase* on medium and hard problems as edits chasing a wrong answer introduce new crashes.

V. DISCUSSION

a) A claim ladder for this class of studies: The main result is operational: taxonomy studies should state the evidence

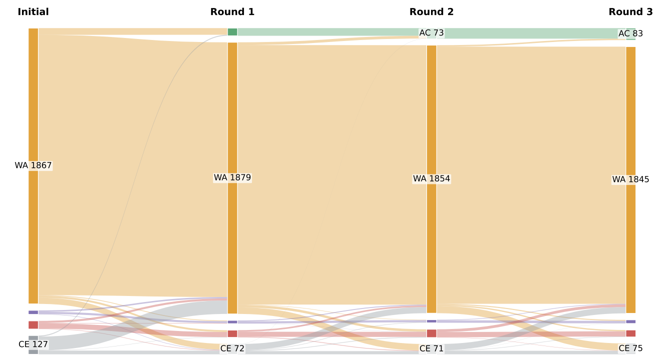

Fig. 4. Verdict transitions of all 2,076 proxy-model bugs across the three self-reflection rounds. The accepted (green) band grows only $0 \to 53 \to 73 \to 83$; wrong answers dominate and persist, and runtime/time-limit failures never resolve.

level of each claim. Sandbox verdicts support outcome and repair dynamics directly. Validated family labels support triage statements such as "most failures look mathematical/design-like." Small source-level shifts require a human gold set, labeler-bias analysis, and problem-level clustering control; in our data they remain screens, not explanations. Root-cause claims require failing-test or reference evidence. We do not validate this ladder universally; we offer it as a transferable checklist and show one full instantiation. It explains why this case study stops at a coarse math/design concentration plus a separate syntax/compile gap.

b) Public-example retry is not a safety net: In our weak-feedback setting, reflection never repairs structural failures and barely shifts the diagnostic-label profile. Pipelines that rely on retrying after public examples need stronger external signals—adversarial tests, complexity analysis, or human review—for the strategic failures that dominate this corpus.

c) Evaluation and measurement: Pass@k is not enough once the remaining failures are algorithmic and small tests miss them. A taxonomy-grounded profile is a useful complementary axis, but RQ1 shows that it must be read carefully: a substantial-agreement judge can still be too coarse to adjudicate small distributional differences, and first-failing evidence can change the family label entirely. Reliability must be established at the granularity of the claim, not only by an aggregate κ; our own coarse claim therefore rests on the extra audit decomposition, not only on full-corpus judge labels.

VI. THREATS TO VALIDITY

We organize threats following standard empirical-SE guidance [52], [53].

Construct validity. The human acceptance "rate" is CodeContests' correct:incorrect ratio, not a real success rate; we compare only within-source *trends* with difficulty. Taxonomy labeling is subjective, so we use two provenance-blind annotators, adjudication, and report inter-annotator $\kappa = 0.80$ as the reliability ceiling. Common-evidence labels are diagnostic hypotheses, not demonstrated root causes; the 120-item

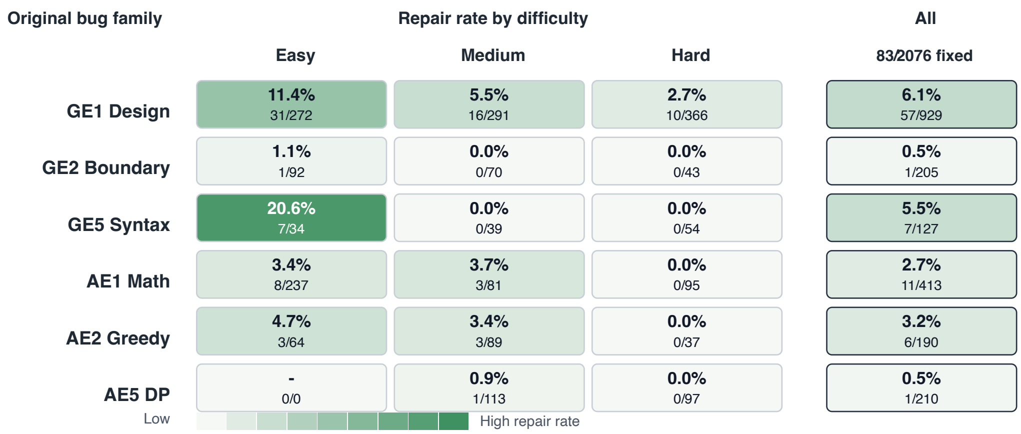

Fig. 5. Taxonomy-aware repair rate of self-reflection, using the original bug's AI-judge primary family label after family-level validation against the 375-item human gold. Rows show families with at least 100 initial proxy-model bugs (two singleton families omitted). Repairs concentrate in easy syntax/design cases and are nearly absent for hard algorithmic families.

stronger-evidence audit shows that first-failing-test/reference evidence changes 55.0% of family labels. RQ1 also shows sensitivity to taxonomy boundaries and granularity, so we fix a "prefer the most specific leaf" rule and avoid fine causal claims. A remaining threat is that the judge (`gpt-5.4`) and modern model (`gpt-5.4-nano`) share a model family: the judge's excess design labels grow across human/proxy/modern (6%/8%/10%). We therefore treat modern-arm taxonomy shifts as screens, not confirmed differences.

Internal validity. Excluding generated tests for tractability admits occasional weak-test pass-through, which can mislabel a buggy program as correct; this affects all sources symmetrically and so should not bias the source-profile checks. Verdicts and acceptance come from a deterministic, cached sandbox and are not subject to labeling error. The modern model's contamination is uncontrolled by design, so its results are a contrast, not a cutoff-controlled measurement. Contamination can also change the *conditional* diagnostic-label profile: if memorized or easy-to-recall solutions become AC, they leave the non-AC pool whose labels we analyze. This selection effect is another reason we make no taxonomy claim from the modern arm; the proxy model anchors only a cutoff-aligned signal.

External validity. We study C++ Codeforces problems, one PaR-style prompt at temperature 1.0, and two models; this bounds the scope. The proxy-model limit is *structural*: post-2021 problems require a pre-2021 training cutoff, ruling out most stronger code models and motivating contamination-aware benchmarks [7]. Its 3% acceptance is a property of this hidden-test setting, not a statement about current LLMs; many failures may be hopeless attempts, which can inflate broad math/design diagnoses. WA-only and accepted-problem checks reduce, but do not remove, this confound. We therefore claim only a bounded coarse profile plus measurement-limited fine shifts.

Conclusion validity. Bugs cluster within problems and labels are multi-label, so we bootstrap over the 107 problems rather than treat 14,182 bugs as independent. Some intervals exclude zero, but effect sizes are small and estimated with the same judge whose family-level bias is 6–9 points. The 125-bug-per-arm gold is also underpowered for small shifts ($V \lesssim 0.26$). The stronger-evidence audit adds only a coarse check: math/design membership is stable for 70.0% of the audit and moves inward more often than outward (25 vs. 11). Thus the dominant coarse region is stable under our checks, while fine source-level differences remain measurement-limited.

VII. Conclusion

Bug-taxonomy labels are useful only when the claim matches the evidence. In this case study, sandbox verdicts support outcome claims; validated family labels support only coarse triage; and neither common-evidence nor automated labels support fine source-level explanations. The transferable lesson is a claim-calibration checklist: validate the labeler, control sample and problem clustering, audit stronger evidence, and state results at the granularity the instrument can sustain.

VIII. Artifact Availability

All artifacts are available at https://anonymous.4open.science/r/aitaxo: problem metadata, prompts, raw generated/reflected programs, judge labels, human gold labels, stronger-evidence audit packs, cached verdicts, and analysis scripts. Exact proprietary-model regeneration is not required for the reported tables.

References

[1] Y. Li *et al.*, "Competition-level code generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[2] M. Wei, Z. Li, X. Chen, M. Zheng, Z. Qu, C. Yu, S. Chen, and X. Ju, "Evaluating and improving LLM-based competitive program generation," *Information and Software Technology*, vol. 191, p. 107977, 2026.

[3] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
[4] J. Austin *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
[5] D. Hendrycks *et al.*, "Measuring coding challenge competence with APPS," in *NeurIPS Datasets and Benchmarks Track*, 2021.
[6] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[7] N. Jain *et al.*, "LiveCodeBench: Holistic and contamination-free evaluation of large language models for code," *arXiv preprint arXiv:2403.07974*, 2024.
[8] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
[9] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
[10] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
[11] E. Nijkamp *et al.*, "CodeGen: An open large language model for code with multi-turn program synthesis," in *International Conference on Learning Representations (ICLR)*, 2023.
[12] B. Rozière *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.
[13] H. Touvron *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
[14] D. Fried *et al.*, "InCoder: A generative model for code infilling and synthesis," in *International Conference on Learning Representations (ICLR)*, 2023.
[15] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of EMNLP*, 2021.
[16] D. Guo *et al.*, "DeepSeek-Coder: When the large language model meets programming," *arXiv preprint arXiv:2401.14196*, 2024.
[17] X. Hou *et al.*, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, 2024.
[18] T. Y. Zhuo *et al.*, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," *arXiv preprint arXiv:2406.15877*, 2024.
[19] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.
[20] S. Lu *et al.*, "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *NeurIPS Datasets and Benchmarks Track*, 2021.
[21] R. Puri *et al.*, "Project CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks," in *NeurIPS Datasets and Benchmarks Track*, 2021.
[22] S. Kulal *et al.*, "SPoC: Search-based pseudocode to code," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
[23] O. Sainz *et al.*, "NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark," in *Findings of EMNLP*, 2023.
[24] S. Golchin and M. Surdeanu, "Time travel in LLMs: Tracing data contamination in large language models," in *International Conference on Learning Representations (ICLR)*, 2024.
[25] M. Riddell, A. Ni, and A. Cohan, "Quantifying contamination in evaluating code generation capabilities of language models," *arXiv preprint arXiv:2403.04811*, 2024.
[26] I. Magar and R. Schwartz, "Data contamination: From memorization to exploitation," in *Proceedings of ACL*, 2022.
[27] R. Chillarege *et al.*, "Orthogonal defect classification—a concept for in-process measurements," *IEEE Transactions on Software Engineering*, vol. 18, no. 11, pp. 943–956, 1992.
[28] L. Tan, C. Liu, Z. Li, X. Wang, Y. Zhou, and C. Zhai, "Bug characteristics in open source software," *Empirical Software Engineering*, vol. 19, no. 6, pp. 1665–1705, 2014.
[29] R.-M. Karampatsis and C. Sutton, "How often do single-statement bugs occur? The ManySStuBs4J dataset," in *Proceedings of MSR*, 2020, pp. 573–577.
[30] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, "Not all bugs are the same: Understanding, characterizing, and classifying bug types," *Journal of Systems and Software*, vol. 152, pp. 165–181, 2019.
[31] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proceedings of ISSTA*, 2014, pp. 437–440.
[32] C. Le Goues *et al.*, "The ManyBugs and IntroClass benchmarks for automated repair of C programs," *IEEE Transactions on Software Engineering*, vol. 41, no. 12, pp. 1236–1256, 2015.
[33] M. Tufano *et al.*, "An empirical study on learning bug-fixing patches in the wild via neural machine translation," *ACM Transactions on Software Engineering and Methodology*, vol. 28, no. 4, pp. 1–29, 2019.
[34] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768.
[35] M. Monperrus, "Automatic software repair: A bibliography," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–24, 2018.
[36] L. Gazzola, D. Micucci, and L. Mariani, "Automatic software repair: A survey," *IEEE Transactions on Software Engineering*, vol. 45, no. 1, pp. 34–67, 2019.
[37] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *Proceedings of ICSE*, 2023, pp. 1482–1494.
[38] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[39] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," *arXiv preprint arXiv:2304.05128*, 2023.
[40] N. Shinn *et al.*, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[41] T. X. Olausson *et al.*, "Is self-repair a silver bullet for code generation?" in *International Conference on Learning Representations (ICLR)*, 2024.
[42] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[43] Y. Liu *et al.*, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in *Proceedings of EMNLP*, 2023.
[44] P. Wang *et al.*, "Large language models are not fair evaluators," in *Proceedings of ACL*, 2024.
[45] W. G. Cochran, *Sampling Techniques*, 3rd ed. John Wiley & Sons, 1977.
[46] R. V. Krejcie and D. W. Morgan, "Determining sample size for research activities," *Educational and Psychological Measurement*, vol. 30, no. 3, pp. 607–610, 1970.
[47] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
[48] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
[49] M. L. McHugh, "Interrater reliability: The kappa statistic," *Biochemia Medica*, vol. 22, no. 3, pp. 276–282, 2012.
[50] R. Artstein and M. Poesio, "Inter-coder agreement for computational linguistics," *Computational Linguistics*, vol. 34, no. 4, pp. 555–596, 2008.
[51] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 4th ed. Sage, 2018.
[52] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
[53] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.